M1: Approach

Project: Accessible Everyday Services Assistant
Technology Stack: Python, Streamlit, Bhashini APIs

1. Problem Statement and Understanding

Everyday activities such as ordering food, reading menus, shopping, and communicating with service staff can be challenging for people with visual, hearing, or speech impairments. Language barriers and dependence on visual or spoken communication can further limit their independence.

Our objective is to develop an accessible web application that enables users to communicate more easily in restaurants, shops, and other everyday services through speech, text, audio output, and multilingual translation.

2. Idea / Abstract

The Accessible Everyday Services Assistant is a Python-based web application designed to simplify everyday interactions for people with different accessibility needs.

The application will offer three modes:

- **Blind / Low-Vision Mode: Provides spoken instructions and reads information aloud.**
- **Deaf / Hard-of-Hearing Mode: Converts supported speech input into readable text and displays responses as text or captions.**
- **Non-Speaking Mode: Allows users to type requests and display them clearly for service staff.**

Streamlit will be used to build the interface, while Bhashini APIs will support available speech recognition, translation, and text-to-speech features. The initial prototype will demonstrate common scenarios, such as reading a menu aloud, translating a request, and displaying a typed order.

3. High-Level Approach and Architecture

The application will follow a modular pipeline:

User → Streamlit Dashboard → Input Processing → Bhashini APIs → Response Processing → Accessible Output

1. **Interface: Users select an accessibility mode through a simple dashboard with spoken guidance where supported.**
2. **Input: Users provide speech or typed text, depending on their needs.**

3. **Processing: Python manages the input and calls Bhashini APIs for supported speech recognition, translation, or speech synthesis.**
4. **Output: Information is delivered as text, captions, or audio according to the selected mode.**
5. **Error Handling: Failed API requests or unsupported languages trigger clear messages and alternative input options.**

The design will prioritize simple navigation, readable content, keyboard accessibility, and alternatives to visual- or audio-only interactions.

4. Planned Testing Methodology

- **Functional Testing: Verify mode selection, input handling, API integration, and output generation.**
- **Accessibility Testing: Check keyboard navigation, screen-reader compatibility, readable text, captions, and spoken instructions.**
- **Language Testing: Test supported speech recognition, translation, and text-to-speech using common service-related phrases.**
- **Scenario Testing: Simulate ordering food, understanding menu information, and communicating requests to service staff.**
- **Error Testing: Test unclear speech, empty inputs, network failures, and unsupported language combinations.**
- **Usability Testing: Gather feedback from accessibility-aware testers and improve the interface based on identified barriers.**

Expected Outcome: A working prototype demonstrating how accessible interaction modes and multilingual speech technologies can make everyday service interactions more inclusive and convenient.